Unit V:

Neural Networks:

1. Introduction
2. Perceptron Learning
3. Backpropagation
4. Training and Validation
5. Parameter Estimation – MLE, MAP, Bayesian

Neural Networks:

1. Introduction:

1. **Structure**: A neural network consists of interconnected units called **neurons**. These neurons send signals to one another. While individual neurons are simple, when many of them work together in a network, they can perform complex tasks.
2. **Layers**: A typical neural network has three main layers:
    - **Input Layer**: Receives input data.
    - **Hidden Layer(s)**: One or more intermediate layers that process information.
    - **Output Layer**: Produces the final output or prediction.
3. **Node (Neuron)**: Each node (neuron) in the network has its own associated **weight** and **threshold**. If the output of a node exceeds the specified threshold, it activates and passes data to the next layer.
4. **Activation Function**: After multiplying inputs by their weights and summing them up, the output passes through an **activation function**. If the result exceeds the threshold, the node fires, connecting to the next layer. This process defines the neural network as a **feedforward network**.
5. **Training**: Neural networks learn from training data to improve accuracy over time. Once fine-tuned, they excel in tasks like speech recognition, image classification, and more.

Evolution of Neural Networks

Since the 1940s, there have been a number of noteworthy advancements in the field of neural networks:

- **1940s-1950s: Early Concepts**
  Neural networks began with the introduction of the first mathematical model of artificial neurons by McCulloch and Pitts. But computational constraints made progress difficult.
- **1960s-1970s: Perceptrons**
  This era is defined by the work of Rosenblatt on perceptrons. Perceptrons are single-layer networks whose applicability was limited to issues that could be solved linearly separately.
- **1980s: Backpropagation and Connectionism**
  Multi-layer network training was made possible by Rumelhart, Hinton, and Williams' invention of the backpropagation method. With its emphasis on learning through interconnected nodes, connectionism gained appeal.

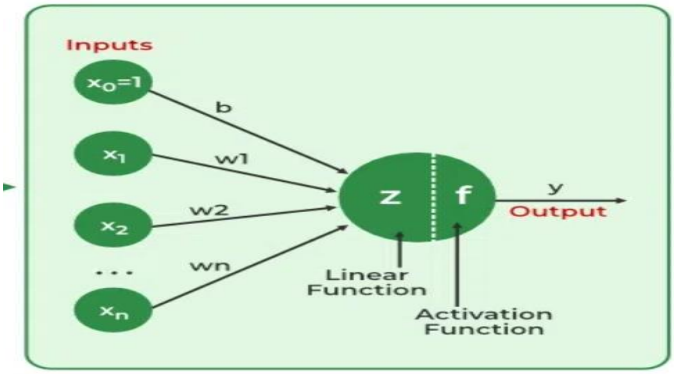
- **1990s: Boom and Winter**
  With applications in image identification, finance, and other fields, neural networks saw a boom. Neural network research did, however, experience a "winter" due to exorbitant computational costs and inflated expectations.
- **2000s: Resurgence and Deep Learning**
  Larger datasets, innovative structures, and enhanced processing capability spurred a comeback. Deep learning has shown amazing effectiveness in a number of disciplines by utilizing numerous layers.
- **2010s-Present: Deep Learning Dominance**
  Convolutional neural networks (CNNs) and recurrent neural networks (RNNs), two deep learning architectures, dominated machine learning. Their power was demonstrated by innovations in gaming, picture recognition, and natural language processing.

What are Neural Networks?

Neural networks extract identifying features from data, lacking pre-programmed understanding. Network components include neurons, connections, weights, biases, propagation functions, and a learning rule. Neurons receive inputs, governed by thresholds and activation functions. Connections involve weights and biases regulating information transfer. Learning, adjusting weights and biases, occurs in three stages: input computation, output generation, and iterative refinement enhancing the network's proficiency in diverse tasks.

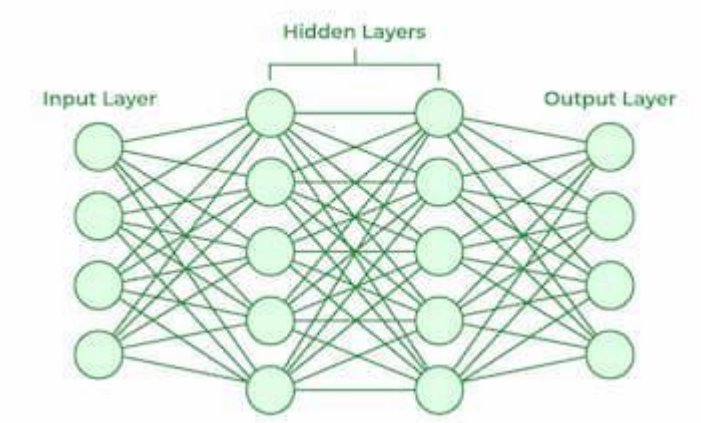
These include:

1. The neural network is simulated by a new environment.
2. Then the free parameters of the neural network are changed as a result of this simulation.
3. The neural network then responds in a new way to the environment because of the changes in its free parameters.



Working of a Neural Network

Neural networks are complex systems that mimic some features of the functioning of the human brain. It is composed of an input layer, one or more hidden layers, and an output layer made up of layers of artificial neurons that are coupled. The two stages of the basic process are called backpropagation and forward propagation.



Forward Propagation

- **Input Layer:** Each feature in the input layer is represented by a node on the network, which receives input data.
- **Weights and Connections:** The weight of each neuronal connection indicates how strong the connection is. Throughout training, these weights are changed.
- **Hidden Layers:** Each hidden layer neuron processes inputs by multiplying them by weights, adding them up, and then passing them through an activation function. By doing this, non-linearity is introduced, enabling the network to recognize intricate patterns.
- **Output:** The final result is produced by repeating the process until the output layer is reached.

Backpropagation

- **Loss Calculation:** The network's output is evaluated against the real goal values, and a loss function is used to compute the difference. For a regression problem, the Mean Squared Error (MSE) is commonly used as the cost function.

Loss Function: $$MSE = \frac{1}{n}\Sigma_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Gradient Descent:** Gradient descent is then used by the network to reduce the loss. To lower the inaccuracy, weights are changed based on the derivative of the loss with respect to each weight.

- **Adjusting weights:** The weights are adjusted at each connection by applying this iterative process, or backpropagation, backward across the network.
- **Training:** During training with different data samples, the entire process of forward propagation, loss calculation, and backpropagation is done iteratively, enabling the network to adapt and learn patterns from the data.
- **Actvation Functions:** Model non-linearity is introduced by activation functions like the rectified linear unit (ReLU) or sigmoid. Their decision on whether to "fire" a neuron is based on the whole weighted input.

Types of Neural Networks

There are *seven* types of neural networks that can be used.

- **Feedforward Neteworks:** A feedforward neural network is a simple artificial neural network architecture in which data moves from input to output in a single direction. It has input, hidden, and output layers; feedback loops are absent. Its straightforward architecture makes it appropriate for a number of applications, such as regression and pattern recognition.
- **Multilayer Perceptron (MLP):** MLP is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers, and an output layer. It uses nonlinear activation functions.
- **Convolutional Neural Network (CNN):** A Convolutional Neural Network (CNN) is a specialized artificial neural network designed for image processing. It employs convolutional layers to automatically learn hierarchical features from input images, enabling effective image recognition and classification. CNNs have revolutionized computer vision and are pivotal in tasks like object detection and image analysis.
- **Recurrent Neural Network (RNN):** An artificial neural network type intended for sequential data processing is called a Recurrent Neural

Network (RNN). It is appropriate for applications where contextual dependencies are critical, such as time series prediction and natural language processing, since it makes use of feedback loops, which enable information to survive within the network.

- **Long Short-Term Memory (LSTM):** LSTM is a type of RNN that is designed to overcome the vanishing gradient problem in training RNNs. It uses memory cells and gates to selectively read, write, and erase information.

Advantages of Neural Networks

Neural networks are widely used in many different applications because of their many benefits:

- **Adaptability:** Neural networks are useful for activities where the link between inputs and outputs is complex or not well defined because they can adapt to new situations and learn from data.
- **Pattern Recognition:** Their proficiency in pattern recognition renders them efficacious in tasks like as audio and image identification, natural language processing, and other intricate data patterns.
- **Parallel Processing:** Because neural networks are capable of parallel processing by nature, they can process numerous jobs at once, which speeds up and improves the efficiency of computations.
- **Non-Linearity:** Neural networks are able to model and comprehend complicated relationships in data by virtue of the non-linear activation functions found in neurons, which overcome the drawbacks of linear models.

Disadvantages of Neural Networks

Neural networks, while powerful, are not without drawbacks and difficulties:

- **Computational Intensity:** Large neural network training can be a laborious and computationally demanding process that demands a lot of computing power.
- **Black box Nature:** As “black box” models, neural networks pose a problem in important applications since it is difficult to understand how they make decisions.
- **Overfitting:** Overfitting is a phenomenon in which neural networks commit training material to memory rather than identifying patterns in the

data. Although regularization approaches help to alleviate this, the problem still exists.

- **Need for Large datasets:** For efficient training, neural networks frequently need sizable, labeled datasets; otherwise, their performance may suffer from incomplete or skewed data.

2.Perceptron Learning

In Machine Learning and Artificial Intelligence, Perceptron is the most commonly used term for all folks. It is the primary step to learn Machine Learning and Deep Learning technologies, which consists of a set of weights, input values or scores, and a threshold. ***Perceptron is a building block of an Artificial Neural Network***. Initially, in the mid of 19th century, **Mr. Frank Rosenblatt** invented the Perceptron for performing certain calculations to detect input data capabilities or business intelligence. Perceptron is a linear Machine Learning algorithm used for supervised learning for various binary classifiers. This algorithm enables neurons to learn elements and processes them one by one during preparation. In this tutorial, "Perceptron in Machine Learning," we will discuss in-depth knowledge of Perceptron and its basic functions in brief. Let's start with the basic introduction of Perceptron.

What is the Perceptron model in Machine Learning?

Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, ***Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence***.

Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**

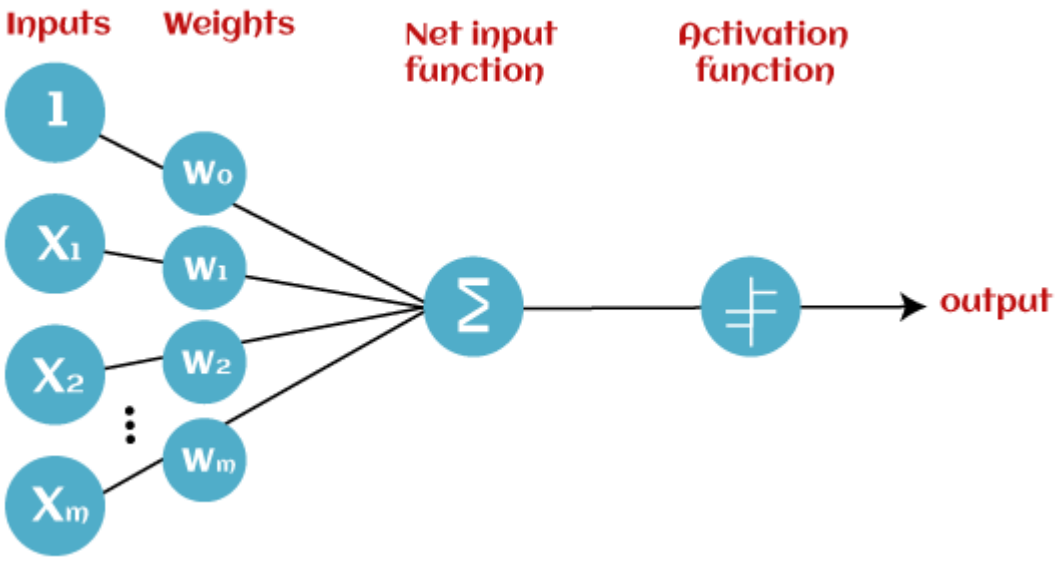
What is Binary classifier in Machine Learning?

In Machine Learning, binary classifiers are defined as the function that helps in deciding whether input data can be represented as vectors of numbers and belongs to some specific class.

Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a ***classification algorithm that can predict linear predictor function in terms of weight and feature vectors.***

Basic Components of Perceptron

Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:



- **Input Nodes or Input Layer:**

This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

- **Wight and Bias:**

Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

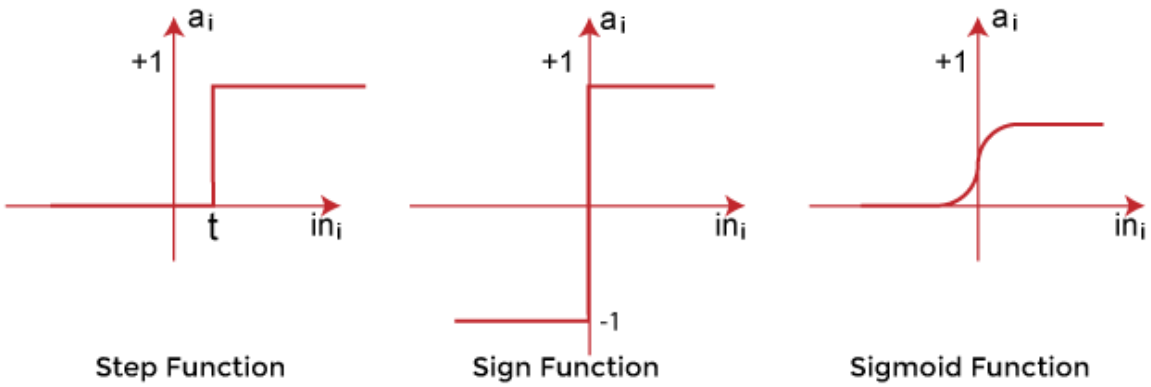
- **Activation Function:**

These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

Types of Activation functions:

- Sign function

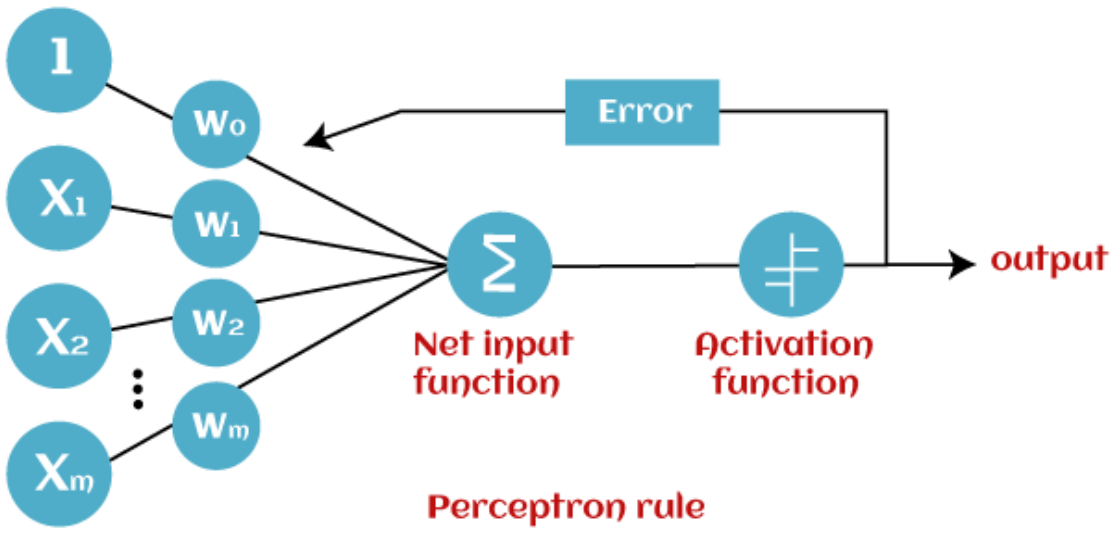
- Step function, and
- Sigmoid function



The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.

How does Perceptron work?

In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named input values (Input nodes), weights and Bias, net sum, and an activation function. The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the **step function** and is represented by **'f'**.



This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

Perceptron model works in two important steps as follows:

Step-1

In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$\sum wi*xi = x1*w1 + x2*w2 + \ldots wn*xn$

Add a special term called **bias 'b'** to this weighted sum to improve the model's performance.

$\sum \mathbf{wi*xi + b}$

Step-2

In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

$\mathbf{Y = f(\sum wi*xi + b)}$

Types of Perceptron Models

Based on the layers, Perceptron models are divided into two types. These are as follows:

1. Single-layer Perceptron Model
2. Multi-layer Perceptron model

Single Layer Perceptron Model:

This is one of the easiest Artificial neural networks (ANN) types. A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.

In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters. Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

If the outcome is same as pre-determined or threshold value, then the performance of this model is stated as satisfied, and weight demand does not change. However, this model consists of a few discrepancies triggered when multiple weight inputs values are fed into the model. Hence, to find desired output and minimize errors, some changes should be necessary for the weights input.

"Single-layer perceptron can learn only linearly separable patterns."

Multi-Layered Perceptron Model:

Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a greater number of hidden layers.

The multi-layer perceptron model is also known as the Backpropagation algorithm, which executes in two stages as follows:

- **Forward Stage:** Activation functions start from the input layer in the forward stage and terminate on the output layer.
- **Backward Stage:** In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

Hence, a multi-layered perceptron model has considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.

A multi-layer perceptron model has greater processing power and can process linear and non-linear patterns. Further, it can also implement logic gates such as AND, OR, XOR, NAND, NOT, XNOR, NOR.

Advantages of Multi-Layer Perceptron:

- A multi-layered perceptron model can be used to solve complex non-linear problems.

- It works well with both small and large input data.
- It helps us to obtain quick predictions after the training.
- It helps to obtain the same accuracy ratio with large as well as small data.

Disadvantages of Multi-Layer Perceptron:

- In Multi-layer perceptron, computations are difficult and time-consuming.
- In multi-layer Perceptron, it is difficult to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of the training.

Perceptron Function

Perceptron function "f(x)" can be achieved as output by multiplying the input 'x' with the learned weight coefficient 'w'.

Mathematically, we can express it as follows:

f(x)=1; if w.x+b>0

otherwise, f(x)=0

- 'w' represents real-valued weights vector
- 'b' represents the bias
- 'x' represents a vector of input x values.

Characteristics of Perceptron

The perceptron model has the following characteristics.

1. Perceptron is a machine learning algorithm for supervised learning of binary classifiers.
2. In Perceptron, the weight coefficient is automatically learned.
3. Initially, weights are multiplied with input features, and the decision is made whether the neuron is fired or not.
4. The activation function applies a step rule to check whether the weight function is greater than zero.

5. The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
6. If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

Limitations of Perceptron Model

A perceptron model has limitations as follows:

- The output of a perceptron can only be a binary number (0 or 1) due to the hard limit transfer function.
- Perceptron can only be used to classify the linearly separable sets of input vectors. If input vectors are non-linear, it is not easy to classify them properly.

Future of Perceptron

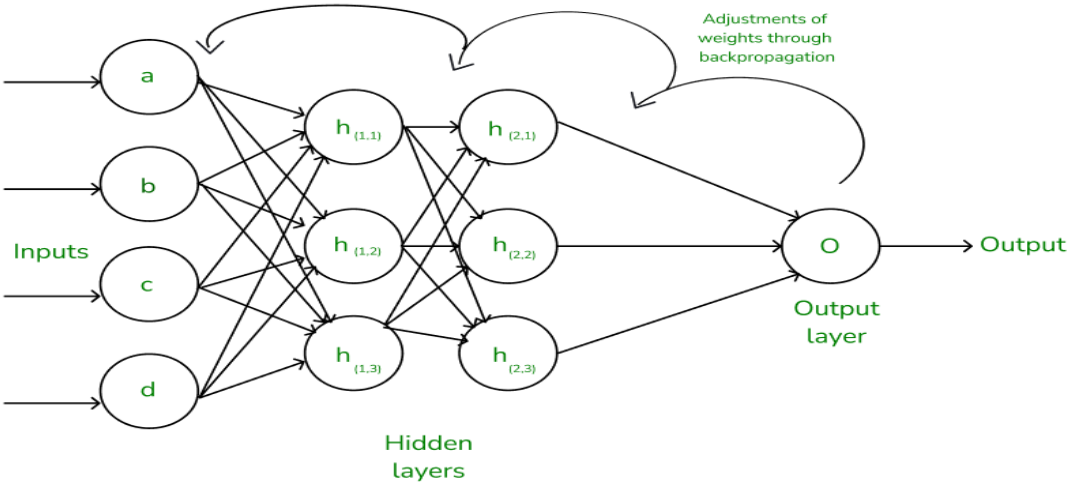
The future of the Perceptron model is much bright and significant as it helps to interpret data by building intuitive patterns and applying them in the future. Machine learning is a rapidly growing technology of Artificial Intelligence that is continuously evolving and in the developing phase; hence the future of perceptron technology will continue to support and facilitate analytical behavior in machines that will, in turn, add to the efficiency of computers.

The perceptron model is continuously becoming more advanced and working efficiently on complex problems with the help of artificial neurons.

3.Backpropagation

- In machine learning, backpropagation is an effective algorithm used to train artificial neural networks, especially in feed-forward neural networks.
- Backpropagation is an iterative algorithm, that helps to minimize the cost function by determining which weights and biases should be adjusted. During every epoch, the model learns by adapting the weights and biases to minimize the loss by moving down toward the gradient of the error. Thus, it involves the two most popular optimization algorithms, such as gradient descent or stochastic gradient descent.

- Computing the gradient in the backpropagation algorithm helps to minimize the cost function and it can be implemented by using the mathematical rule called chain rule from calculus to navigate through complex layers of the neural network.



Advantages of Using the Backpropagation Algorithm in Neural Networks

Backpropagation, a fundamental algorithm in training neural networks, offers several advantages that make it a preferred choice for many machine learning tasks. Here, we discuss some key advantages of using the backpropagation algorithm:

1. **Ease of Implementation:** Backpropagation does not require prior knowledge of neural networks, making it accessible to beginners. Its straightforward nature simplifies the programming process, as it primarily involves adjusting weights based on error derivatives.

2. **Simplicity and Flexibility:** The algorithm's simplicity allows it to be applied to a wide range of problems and network architectures. Its flexibility makes it suitable for various scenarios, from simple feedforward networks to complex recurrent or convolutional neural networks.

3. **Efficiency:** Backpropagation accelerates the learning process by directly updating weights based on the calculated error derivatives. This efficiency is particularly advantageous in training deep neural networks, where learning features of a function can be time-consuming.

4. **Generalization:** Backpropagation enables neural networks to generalize well to unseen data by iteratively adjusting weights during training. This generalization ability is crucial for developing models that can make accurate predictions on new, unseen examples.

5. **Scalability:** Backpropagation scales well with the size of the dataset and the complexity of the network. This scalability makes it suitable for large-scale machine learning tasks, where training data and network size are significant factors.

Working of Backpropagation Algorithm

The Backpropagation algorithm works by two different passes, they are:

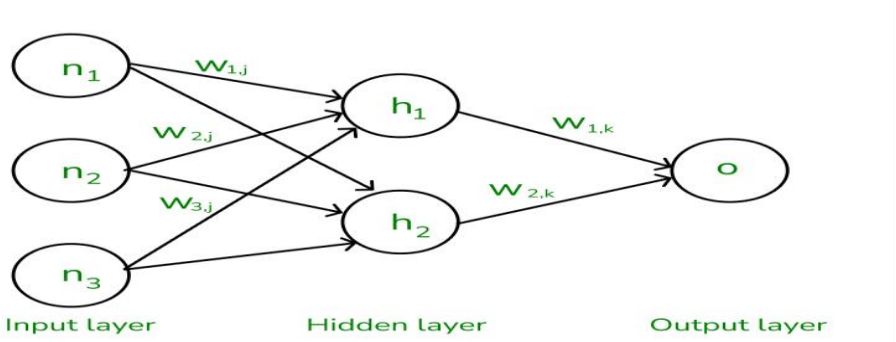
- Forward pass
- Backward pass

How does Forward pass work?

- In forward pass, initially the input is fed into the input layer. Since the inputs are raw data, they can be used for training our neural network.

- The inputs and their corresponding weights are passed to the hidden layer. The hidden layer performs the computation on the data it receives. If there are two hidden layers in the neural network, for instance, consider the illustration fig(a), h1 and h2 are the two hidden layers, and the output of h1 can be used as an input of h2. Before applying it to the activation function, the bias is added.

- To the weighted sum of inputs, the activation function is applied in the hidden layer to each of its neurons. One such activation function that is commonly used is ReLU can also be used, which is responsible for returning the input if it is positive otherwise it returns zero. By doing this so, it introduces the non-linearity to our model, which enables the network to learn the complex relationships in the data. And finally, the weighted outputs from the last hidden layer are fed into the output to compute the final prediction, this layer can also use the activation function called the softmax function which is responsible for converting the weighted outputs into probabilities for each class.



The forward pass using weights and biases

How does backward pass work?

- In the backward pass process shows, the error is transmitted back to the network which helps the network, to improve its performance by learning and adjusting the internal weights.

- To find the error generated through the process of forward pass, we can use one of the most commonly used methods called mean squared error which calculates the difference between the predicted output and desired output. The formula for mean squared error is: $Meansquarederror=(predictedoutput–actualoutput)2$*Meansquarederror*=(*predictedoutput–actualoutput*)2

- Once we have done the calculation at the output layer, we then propagate the error backward through the network, layer by layer.

- The key calculation during the backward pass is determining the gradients for each weight and bias in the network. This gradient is responsible for telling us how much each weight/bias should be adjusted to minimize the error in the next forward pass. The chain rule is used iteratively to calculate this gradient efficiently.

- In addition to gradient calculation, the activation function also plays a crucial role in backpropagation, it works by calculating the gradients with the help of the derivative of the activation function.

Python program for backpropagation

Here’s a simple implementation of feedforward neural network with backpropagation in Python:

1. **Neural Network Initialization**: The NeuralNetwork class is initialized with parameters for the input size, hidden layer size, and output size. It also initializes the weights and biases with random values.

2. **Sigmoid Activation Function**: The sigmoid method implements the sigmoid activation function, which squashes the input to a value between 0 and 1.

3. **Sigmoid Derivative**: The sigmoid_derivative method calculates the derivative of the sigmoid function. It computes the gradients of the loss function with respect to weights.

4. **Feedforward Pass**: The feedforward method calculates the activations of the hidden and output layers based on the input data and current weights and biases. It uses matrix multiplication to propagate the inputs through the network.

5. **Backpropagation**: The backward method performs the backpropagation algorithm. It calculates the error at the output layer and propagates it back through the network to update the weights and biases using gradient descent.

6. **Training the Neural Network**: The train method trains the neural network using the specified number of epochs and learning rate. It iterates through the training data, performs the feedforward and backward passes, and updates the weights and biases accordingly.

7. **XOR Dataset**: The XOR dataset (X) is defined, which contains input pairs that represent the XOR operation, where the output is 1 if exactly one of the inputs is 1, and 0 otherwise.

8. **Testing the Trained Model**: After training, the neural network is tested on the XOR dataset (X) to see how well it has learned the XOR function. The predicted outputs are printed to the console, showing the neural network's predictions for each input pair.

4.Training and Validation

What is Training data?

Testing data is used to determine the performance of the trained model, whereas training data is used to train the machine learning model. Training data is the power that supplies the model in machine learning, it is larger

than testing data. Because more data helps to more effective predictive models. When a machine learning algorithm receives data from our records, it recognizes patterns and creates a decision-making model.

Algorithms allow a company's past experience to be used to make decisions. It analyzes all previous cases and their results and, using this data creates models to score and predict the outcome of current cases. The more data ML models have access to, the more reliable their predictions get over time.

What is Testing Data?

You will need unknown information to test your machine learning model after it was created (using your training data). This data is known as testing data, and it may be used to assess the progress and efficiency of your algorithms' training as well as to modify or optimize them for better results.

- Showing the original set of data.
- Be large enough to produce reliable projections

This dataset needs to be "unseen" and recent. This is because the training data was already "learned" by your model. You can decide if it is operating successfully or when it need more training data to fulfill your standards by observing how it performs on fresh test data. Test data provides as a last, real check if an unknown dataset was correctly trained by the machine learning algorithm.

Difference between Training data and Testing data

Features	Training Data	Testing Data
Purpose	The machine-learning model is trained using training data. The more training data a model has, the more accurate predictions it can make.	Testing data is used to evaluate the model's performance.

Features	Training Data	Testing Data
Exposure	By using the training data, the model can gain knowledge and become more accurate in its predictions.	Until evaluation, the testing data is not exposed to the model. This guarantees that the model cannot learn the testing data by heart and produce flawless forecasts.
Distribution	This training data distribution should be similar to the distribution of actual data that the model will use.	The distribution of the testing data and the data from the real world differs greatly.
Use	To stop overfitting, training data is utilized.	By making predictions on the testing data and comparing them to the actual labels, the performance of the model is assessed.
Size	Typically larger	Typically smaller

Why do we need Training data and Testing data

Training data teaches a machine learning model how to behave, whereas testing data assesses how well the model has learned.

- **Training Data:** The machine learning model is taught how to generate predictions or perform a specific task using training data. Since it is usually identified, every data point's output from the model is known. In order to provide predictions, the model must first learn to recognize patterns in the data. Training data can be compared to a student's textbook when learning

a new subject. The learner learns by reading the text and completing the tasks, and the book offers all the knowledge they require.

- **Testing Data:** The performance of the machine learning model is measured using testing data. Usually, it is labeled and distinct from the training set. This indicates that for every data point, the model's result is unknown. On the testing data, the model's accuracy in predicting outcomes is assessed. Testing data is comparable to the exam a student takes to determine how well-versed in a subject they are. The test asks questions that the student must respond to, and the test results are used to gauge the student's comprehension.

Training and Validation in Machine Learning

Training

Training is the process where a machine learning model learns from a dataset. This dataset, known as the training set, contains input-output pairs where the output (or label) is known. The model adjusts its parameters to minimize the error between its predictions and the actual labels in the training set.

Key Points:

- **Objective:** Minimize the prediction error by adjusting model parameters.
- **Dataset:** Training set, with known input-output pairs.
- **Process:** Iterative, involving techniques like gradient descent.

Validation

Validation is the process of evaluating the model's performance on a separate dataset, called the validation set. This helps in tuning the model's hyperparameters and checking for issues like overfitting without using the test set, which is reserved for final evaluation.

Key Points:

- **Objective:** Assess model performance and tune hyperparameters.
- **Dataset:** Validation set, separate from the training set.
- **Process:** Typically performed after each training iteration or epoch.

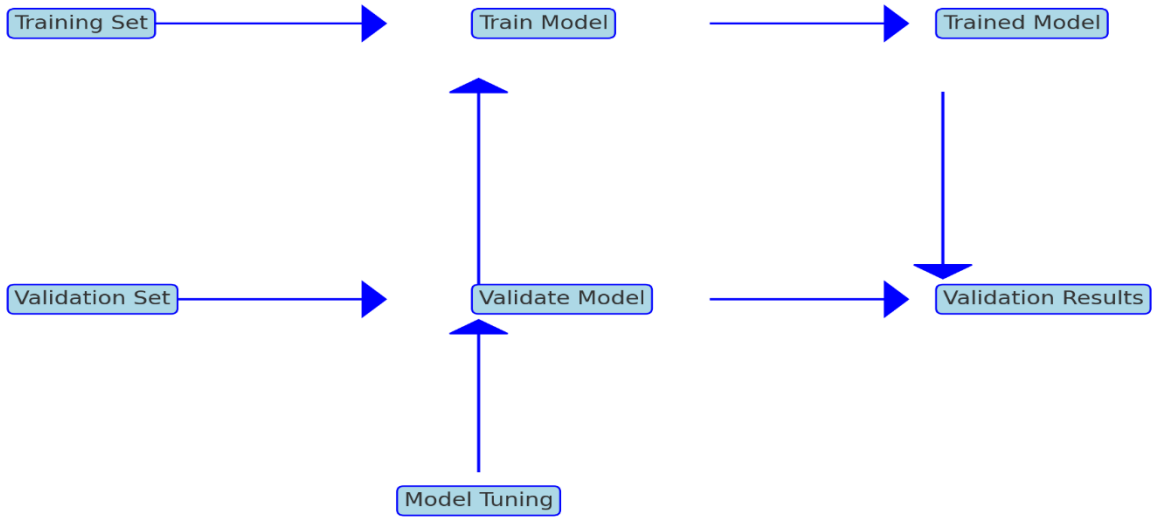
Differences Between Training and Validation

1. **Purpose:**

    - **Training:** To adjust model parameters to minimize the error on the training data.
    - **Validation:** To evaluate model performance and tune hyperparameters, ensuring the model generalizes well to unseen data.
2. **Data:**
    - **Training:** Uses the training dataset.
    - **Validation:** Uses the validation dataset, which is distinct from the training data.
3. **Process:**
    - **Training:** Involves learning from the data, often through backpropagation and optimization algorithms.
    - **Validation:** Involves monitoring performance metrics and making decisions about model adjustments.
4. **Frequency:**
    - **Training:** Continuous, until the model converges or training criteria are met.
    - **Validation:** Periodic, often after each epoch or a set number of iterations.
5. **Impact on Model:**
    - **Training:** Directly influences the model's weights and biases.
    - **Validation:** Influences decisions on model tuning, such as hyperparameter settings and stopping criteria.
6. **Outcome:**
    - **Training:** A model that performs well on the training data.
    - **Validation:** Insights into the model's ability to generalize, guiding improvements to prevent overfitting and underfitting.

Example Workflow

1. **Split Data:** Divide the dataset into training, validation, and test sets (e.g., 70% training, 20% validation, 10% test).
2. **Train Model:** Use the training set to train the model.
3. **Validate Model:** Use the validation set to evaluate performance and adjust hyperparameters.
4. **Final Evaluation:** After tuning, use the test set to assess the model's final performance.



training and validation process

Training Phase:

- **Training Set:** The dataset used to train the model.
- **Train Model:** The process where the model learns from the training data.
- **Trained Model:** The outcome after the model has been trained on the training data.

Validation Phase:

- **Validation Set:** The dataset used to evaluate the model's performance.
- **Validate Model:** The process of assessing the trained model using the validation data.
- **Validation Results:** The outcomes of the validation process, used to tune the model.

Model Tuning:

- Based on the validation results, the model is tuned (e.g., adjusting hyperparameters) to improve its performance.

5 Parameter Estimation – MLE, MAP, Bayesian

Parameter estimation in neural networks involves determining the best set of parameters (weights and biases) for the model based on the training data. Three common approaches to parameter estimation are Maximum Likelihood Estimation (MLE), Maximum A Posteriori (MAP) estimation, and Bayesian estimation. Here's a brief overview of each:

1. Maximum Likelihood Estimation (MLE)

- **Concept**: MLE aims to find the parameters that maximize the likelihood function, which measures how well the model explains the observed data.
- **Formula**: Given a dataset $D = \{x_i, y_i\}$, the likelihood of the parameters θ is $P(D|\theta) = \prod_i P(y_i|x_i, \theta)$. The MLE $\hat{\theta}_{MLE}$ is found by maximizing this likelihood: $\hat{\theta}_{MLE} = \arg\max_\theta P(D|\theta)$.
- **Application**: In neural networks, this involves adjusting the weights and biases to maximize the probability of the observed data given the model's predictions.

2) Maximum A Posteriori (MAP) Estimation

- **Concept**: MAP estimation extends MLE by incorporating prior knowledge about the parameters. It maximizes the posterior distribution of the parameters given the data.
- **Formula**: The posterior distribution is given by Bayes' theorem: $P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$. The MAP estimate $\hat{\theta}_{MAP}$ is found by maximizing this posterior: $\hat{\theta}_{MAP} = \arg\max_\theta P(\theta|D) = \arg\max_\theta P(D|\theta)P(\theta)$.
- **Application**: In neural networks, this involves not only maximizing the likelihood but also considering a prior distribution over the parameters, which can act as a regularizer.

3. Bayesian Estimation

- **Concept**: Bayesian estimation provides a full probabilistic description of the parameters. Instead of finding a single best estimate, it computes the entire posterior distribution of the parameters given the data.
- **Formula**: The posterior distribution is $P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$. Bayesian inference uses this distribution to make predictions, often by averaging over many possible parameter values.
- **Application**: In neural networks, Bayesian methods can involve techniques like Bayesian Neural Networks (BNNs) where weights are treat as distributions, and inference is performed over these distributions rather than point estimates.

Summary of Differences

- **MLE**: Focuses solely on the data, ignoring any prior information.
- **MAP**: Balances data and prior information, offering a compromise between data fit and prior beliefs.
- **Bayesian**: Provides a comprehensive approach by considering the full posterior distribution, leading to a more robust estimation that accounts for uncertainty in the parameters.

Practical Considerations in Neural Networks

- **Computational Complexity**: Bayesian methods are generally more computationally intensive than MLE and MAP due to the need to sample or approximate the posterior distribution.
- **Regularization**: MAP estimation introduces regularization naturally through the prior distribution, which can help in preventing overfitting.
- **Uncertainty Quantification**: Bayesian approaches allow for uncertainty quantification in predictions, which can be particularly valuable in applications where understanding confidence in predictions is important.

Q:
How does Bayesian parameter estimation differ from MLE and MAP, and what are its advantages in neural network modelling?

Bayesian parameter estimation differs from Maximum Likelihood Estimation (MLE) and Maximum A Posteriori (MAP) estimation in several key ways, and it offers distinct advantages for neural network modeling. Here's a detailed comparison:

Differences between Bayesian Estimation, MLE, and MAP

1. **Parameter Estimation Approach**:
   - **MLE**: Finds the parameter values that maximize the likelihood function based solely on the observed data. It doesn't consider any prior information about the parameters.

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} P(D|\theta)$$

MAP: Extends MLE by incorporating prior beliefs about the parameters. It maximizes the posterior distribution, which is the product of the likelihood and the prior.

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} P(\theta|D) = \arg\max_{\theta} P(D|\theta)P(\theta)$$

Bayesian: Instead of finding a single best estimate, Bayesian estimation computes the full posterior distribution of the parameters given the data. This approach considers both the likelihood and the prior, and integrates over all possible parameter values to make predictions.

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Output:

- **MLE**: Provides point estimates of parameters.
- **MAP**: Also provides point estimates but incorporates prior knowledge, which can act as a regularizer.
- **Bayesian**: Provides a distribution over parameters, representing uncertainty about the parameter values.

Uncertainty Quantification:

- **MLE and MAP**: Do not provide a direct measure of uncertainty in the parameter estimates.
- **Bayesian**: The posterior distribution inherently represents the uncertainty in the parameter estimates.

Advantages of Bayesian Parameter Estimation in Neural Network Modeling

1. **Uncertainty Quantification**:
   - Bayesian methods provide a natural way to quantify uncertainty in both the parameters and the predictions. This is particularly useful in applications where understanding the confidence in predictions is crucial (e.g., medical diagnosis, autonomous driving).
2. **Regularization**:
   - The prior distribution in Bayesian estimation acts as a regularizer, helping to prevent overfitting, especially in cases where data is scarce. This is similar to MAP, but Bayesian methods take this a step further by fully integrating over the posterior.
3. **Robustness**:
   - Bayesian models are generally more robust to overfitting compared to MLE, as they incorporate prior beliefs and account for uncertainty.
4. **Prediction Averaging**:
   - Bayesian methods can perform model averaging, where predictions are averaged over many possible models (parameter settings) weighted by their posterior probability. This often leads to better generalization performance compared to single point estimates from MLE or MAP.
5. **Flexibility**:

- Bayesian frameworks are flexible and can naturally incorporate various types of prior knowledge. This can be particularly advantageous in complex neural network models where domain knowledge can significantly improve model performance.

6. **Adaptability to New Data**:
   - Bayesian methods can more easily adapt to new data through the updating of the posterior distribution. This is in contrast to MLE and MAP, which may require re-training from scratch or significant adjustments when new data is introduced.

Practical Considerations

- **Computational Complexity**:
  - Bayesian methods can be computationally intensive due to the need to sample from or approximate the posterior distribution, especially in high-dimensional parameter spaces typical in neural networks. Techniques like Markov Chain Monte Carlo (MCMC) or variational inference are often used to address these challenges.
- **Implementation**:
  - Implementing Bayesian neural networks can be more complex compared to standard MLE or MAP-based networks. However, advances in probabilistic programming and software libraries (e.g., Pyro, TensorFlow Probability) are making Bayesian approaches more accessible.

parameter estimation and highlights the differences and advantages of Bayesian estimation over MLE and MAP in neural network modeling.

- Data (D):
  - The starting point for all parameter estimation methods. Represents the observed data used for training the neural network.
- Likelihood $P(D|\theta)$:
  - The probability of the data given the parameters (θ). This is the key component for MLE and MAP.
- MLE (θ_MLE):
  - MLE focuses solely on maximizing the likelihood to find the best parameters. It does not incorporate any prior information.
- Prior $P(\theta)$:

- In MAP and Bayesian estimation, prior information about the parameters is incorporated through the prior distribution.

 MAP (θ_MAP):

- MAP estimation finds parameters that maximize the posterior distribution, which is the product of the likelihood and the prior. It balances fitting the data with prior beliefs about the parameters.

 Bayesian Estimation:

- Bayesian estimation computes the full posterior distribution over the parameters, providing a comprehensive view of uncertainty and incorporating prior knowledge.

 Bayesian Neural Network Predictions:

- Predictions are averaged over the posterior distribution, leading to better uncertainty quantification and robustness against overfitting.